

UR10e Maze Solver

Baaqer Farhat

Andrey Korolev

June 11, 2026

1 Summary

This project makes a UR10e robot arm solve a physical maze on its own. A maze is printed on a sheet of paper and placed on a table in front of the robot. The robot holds a pen, takes a picture of the maze with a wrist-mounted camera, finds the maze in the image, plans a path from the entrance to the exit, and then draws the solution on the paper with the pen.

The motivation is to combine, in one working system, the main topics of the course: computer vision for sensing, kinematics for mapping between task space and joint space, path planning, and feedback control for execution. Each stage has to hand off correct data to the next across multiple coordinate frames, and the full chain has to work on real hardware. An error in any stage directly affects the drawn result.

The scope covers the full pipeline from raw camera image to finished pen drawing:

1. Move the robot to a fixed overhead pose and capture an image of the maze with an Intel RealSense camera.
2. Localize the maze using four AprilTags and rectify the image with a homography.
3. Solve the maze with A* on a grid built from the rectified image, then smooth the path with a spline.
4. Convert the pixel path to metric coordinates, then to robot base coordinates, then to joint space with inverse kinematics.
5. Track the joint trajectory with a model predictive controller (MPC) that streams joint velocity commands to the robot at 75 Hz.

Every stage writes its intermediate results to disk (annotated detections, rectified images, planned paths, joint trajectories, and per-cycle control telemetry). This made it possible to isolate failures to a specific stage instead of guessing. It mattered in practice: most of the debugging effort went into the execution layer (command streaming and feedback rates), and the recorded telemetry is what exposed those problems.

2 System Pipeline

2.1 Image Capture and Maze Localization

The camera is an Intel RealSense mounted on the robot wrist (eye-in-hand). We use its color stream at 1280×720 at 30 fps. Because the camera moves with the arm, every capture is taken from the

same fixed overhead joint pose, so the camera extrinsics (its pose relative to the maze plane) are repeatable from run to run. Four AprilTags are placed around the maze sheet. We detect the tags and use their outermost corners as a quadrilateral that encloses the sheet.

Under the pinhole model, a world point $\mathbf{P} = (X, Y, Z)^T$ projects to image pixel $\mathbf{p} = (u, v, 1)^T$ through the intrinsic matrix K and the extrinsics $(R, \mathbf{t})$:

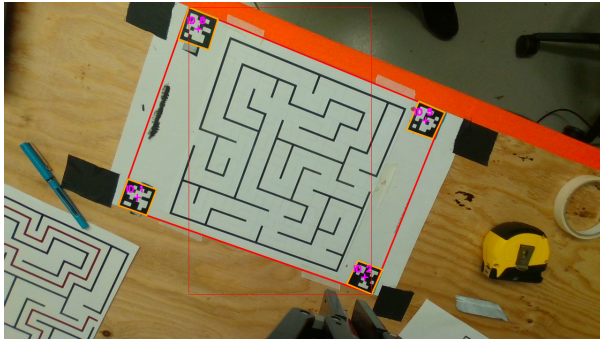
$$\lambda \mathbf{p} = K [R \mid \mathbf{t}] \begin{bmatrix} \mathbf{P} \\ 1 \end{bmatrix}, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (1)$$

The maze lies on a single plane, so we can put the world frame on the paper with $Z = 0$. The third column of R then drops out and the projection collapses to a 3×3 map,

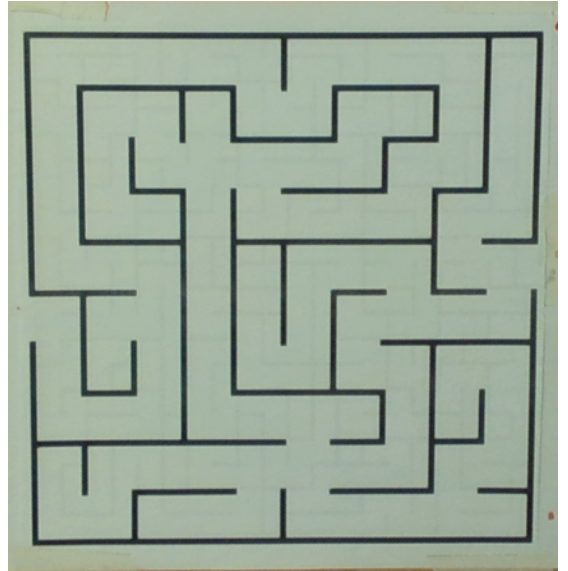
$$\lambda \mathbf{p} = K [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}] \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}, \quad (2)$$

which is a homography. This is the key simplification of the planar setup: the combined effect of the intrinsics and extrinsics is a single invertible 3×3 matrix. We do not need to calibrate K , R , and $\mathbf{t}$ separately. We estimate H directly from point correspondences and undo the perspective in one step.

The camera does not look at the sheet perfectly straight down, so the maze appears under perspective distortion. We remove it by estimating the homography between the detected tag quadrilateral and an axis-aligned rectangle: each correspondence $\mathbf{p} \leftrightarrow \mathbf{p}'$ satisfies $\mathbf{p}' \sim H \mathbf{p}$ (equality up to scale), giving two independent linear equations per point, and the four tag corners fix all eight degrees of freedom of H . Warping the image through H flattens the whole sheet.



(a) Captured image with detected AprilTags and the enclosing quad.



(b) Rectified and cropped maze after the homography.

Figure 1: Maze localization. Left: raw capture with tag detections. Right: the flattened maze used for planning.

After rectification we still need a metric scale. The physical tag side length L is known (42.86 mm). We measure the average tag edge length $\bar{e}$ in rectified pixels and get the scale

$$s = \frac{L}{\bar{e}} \quad \left[\frac{\text{m}}{\text{px}} \right]. \quad (3)$$

Since the homography makes the scale uniform across the sheet, a single scalar fully determines the metric mapping. On our captures $s \approx 0.69 \text{ mm/px}$, which gave a measured maze size of about $291 \times 299 \text{ mm}$. The scale was repeatable to within 0.01% between captures.

2.2 Path Planning with A*

The rectified maze image is turned into a 120×120 grid of nodes. A node is free if enough of the pixels in its cell are brighter than a threshold (paper), and blocked otherwise (wall). Walls are inflated by 6 px before sampling so the planned path keeps a safety margin from the walls; this matters because the pen has physical width.

The maze entrance and exit are found automatically. We scan each side of the image for places where the outer wall line dips inward (a gap). Each candidate gap is then validated with a probe along the inward normal: a real opening has a free corridor leading into the maze, while artifacts from glare, shadows, or the sheet margin have wall pixels just inside them. Only validated openings are kept. A* then searches the grid from the entrance node to the exit node using an octile-distance heuristic.

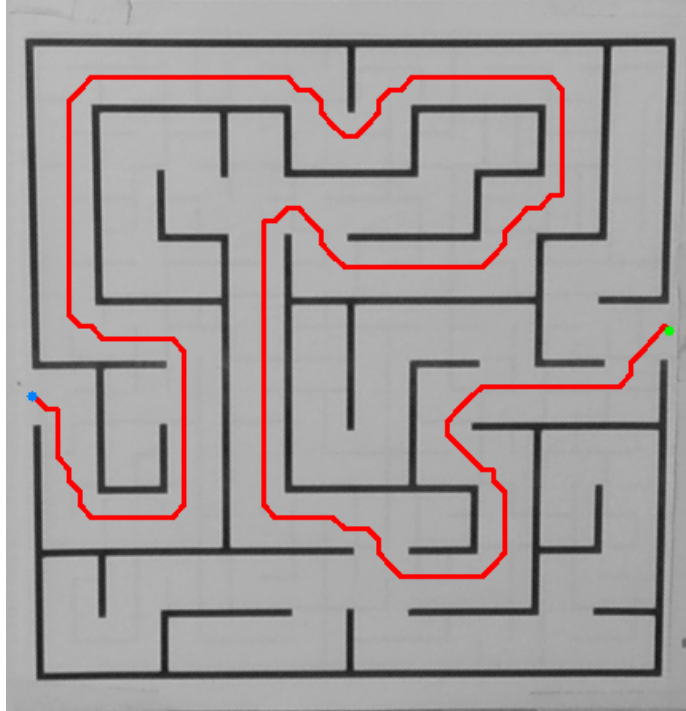


Figure 2: A* solution overlaid on the maze. Green is the entrance (start), blue is the exit (goal).

The raw A* path is piecewise linear and has sharp corners. We smooth it with a spline fitted through every third path node, sampled densely along its length. The spline is what the robot will actually draw.

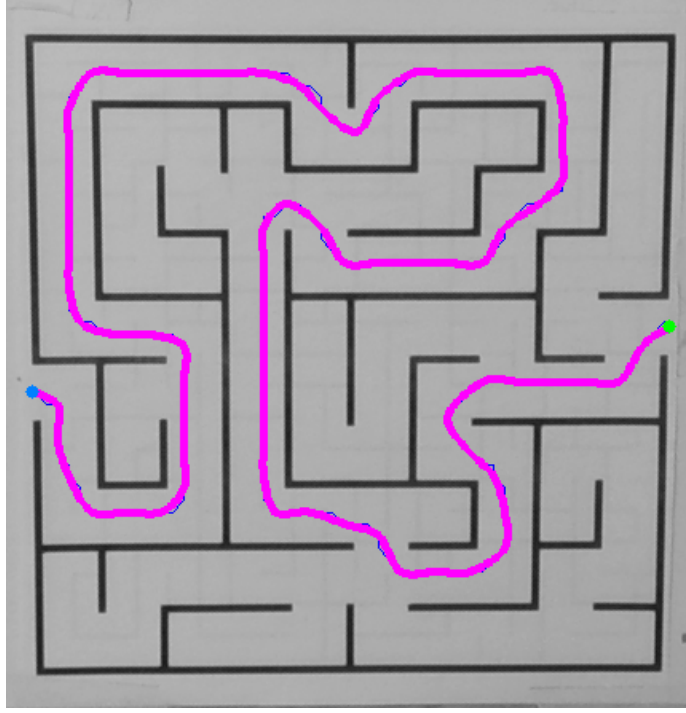


Figure 3: Smoothed spline (magenta) through the A* path. This is the curve sent to the robot.

2.3 Coordinate Frames and Transformations

The pipeline moves every point through four frames: rectified image pixels $\rightarrow$ maze frame (metric, on the paper) $\rightarrow$ anchor tag frame $\rightarrow$ robot base frame. All rigid transforms are homogeneous matrices

$$T = \begin{bmatrix} R & \mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix} \in SE(3), \quad R \in SO(3), \quad \mathbf{p} \in \mathbb{R}^3. \quad (4)$$

A spline point (u, v) in rectified pixels is first scaled to metric maze coordinates using the tag-derived scale,

$$\mathbf{p}_{maze} = \begin{bmatrix} u \, s \\ v \, s \\ 0 \\ 1 \end{bmatrix}, \quad (5)$$

with the maze origin at the top-left corner of the rectified crop and $z = 0$ on the paper. The point is then carried into the robot base frame through the chain

$$\mathbf{p}_{base} = \underbrace{T_{base \leftarrow tag}}_{\text{measured}} \underbrace{T_{tag \leftarrow maze}}_{\text{measured}} \mathbf{p}_{maze}, \quad (6)$$

where $T_{base \leftarrow tag}$ is the pose of the anchor AprilTag in the robot base frame and $T_{tag \leftarrow maze}$ is the offset from that tag to the maze origin; both were measured physically on the workcell. One axis calibration is folded into this chain: the maze image x axis (columns to the right) points along negative base x in our workcell layout, so the first column of $T_{base \leftarrow maze}$ is negated. The drawing height is enforced by replacing z with the measured paper-plane height in the base frame, and the tool orientation is held constant, pen pointing straight down, expressed as a fixed rotation R_{down} .

The robot is commanded at the pen tip, not the flange. With $T_{base \leftarrow tcp}(q) = \text{FK}(q)$ the forward kinematics of the arm and $T_{tcp \leftarrow pen}$ the fixed pen tool transform, the pen pose is

$$T_{base \leftarrow pen}(q) = \text{FK}(q) T_{tcp \leftarrow pen}. \quad (7)$$

For each Cartesian waypoint we build the desired pen pose $T_{base \leftarrow pen}^{des} = (R_{down}, \mathbf{p}_{base})$ and solve

$$\text{FK}(q) = T_{base \leftarrow pen}^{des} T_{tcp \leftarrow pen}^{-1} \quad (8)$$

for q with the closed-form UR10e inverse kinematics. The UR10e IK has up to eight solution branches; we select a consistent elbow-up branch for every waypoint so the arm configuration never flips mid-path. The resulting joint trajectory is validated by running it back through the forward kinematics and checking that the reconstructed pen path matches the planned Cartesian path.

The IK output points are evenly spaced in pixels, but the pixel-to-joint map is nonlinear, so they are *not* evenly spaced in joint space. Since the controller consumes exactly one point per control period Δt , the spacing between points sets the speed. We therefore resample the joint path to uniform joint-space arc length. With total path length $\mathcal{L} = \sum_k \|q_{k+1} - q_k\|$ and a chosen cruise speed v_c , the number of points is

$$N_{pts} = \left\lceil \frac{\mathcal{L}}{v_c \Delta t} \right\rceil, \quad (9)$$

and the trajectory is interpolated at uniform arc-length steps. After this step the commanded joint speed is constant along the whole path, which removed large reference acceleration spikes we measured before the change.

2.4 Model Predictive Control

The robot accepts joint velocity commands, so we model each joint as a single integrator,

$$q_{k+1} = q_k + \Delta t u_k, \quad q, u \in \mathbb{R}^6, \quad (10)$$

with $\Delta t = 1/75$ s. At every control step the MPC solves

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}} \quad & \sum_{t=0}^{N-1} \left[(q_t - q_t^r)^T Q (q_t - q_t^r) + (u_t - u_t^r)^T R (u_t - u_t^r) \right] + (q_N - q_N^r)^T P (q_N - q_N^r) \\ \text{s.t.} \quad & q_{t+1} = q_t + \Delta t u_t, \quad q_0 = q_{meas}, \\ & |u_t| \leq v_{max}, \quad \left| \frac{u_t - u_{t-1}}{\Delta t} \right| \leq a_{max}, \end{aligned} \quad (11)$$

with horizon $N = 6$, velocity limit $v_{max} = 0.6$ rad/s, and acceleration limit $a_{max} = 1.2$ rad/s². The reference q^r is the next $N+1$ points of the joint trajectory. The reference velocity (feedforward) is the discrete derivative of the reference,

$$u_t^r = \frac{\text{wrap}(q_{t+1}^r - q_t^r)}{\Delta t}, \quad (12)$$

where $\text{wrap}(\cdot)$ takes the shortest angular difference so that 2π wraparounds in the IK output do not create velocity spikes. The problem is built once in CasADi as a sequential quadratic program and solved at 75 Hz; only the first input u_0 is sent to the robot each cycle.

2.5 Execution on the Robot

A dedicated thread talks to the robot over the `urx` interface. Static point-to-point motions (moving to the overhead camera pose, approaching the maze start, lowering the pen, and returning home) use blocking `movej` position commands. During the drawing itself, joint positions are read from the robot’s real-time interface at 125 Hz, and the MPC’s velocity command is streamed with `speedj` at 100 Hz. Two execution details turned out to be critical and cost most of the debugging time:

- **Feedback rate.** The default position read updates at only 10 Hz. The 75 Hz controller then sees a frozen measurement for about 7 cycles in a row, builds up error, and overcorrects when the measurement finally jumps. This produced a 10 Hz sawtooth in the command and visible jitter on the arm. Switching to the 125 Hz real-time stream removed it.
- **Streaming acceleration.** Each `speedj` message interrupts the previous one, and the robot re-accelerates toward the new velocity from the deceleration caused by the interruption. With a low acceleration setting the robot only ever reached a fraction of the commanded speed (we measured 18%), which shrank the drawn path. The streaming acceleration must be high enough to reach the commanded velocity within one cycle; it is separate from the a_{max} used inside the MPC, which is what actually keeps the motion smooth.

A safety layer checks every outgoing command: predicted joint positions are tested against joint limits and a link self-collision check before the command is sent.

3 Output

The full system was run end to end on the real robot: capture, localization, planning, and drawing. The robot draws the solution from the entrance to the exit with the pen. A video of the complete run on the real robot is available here:

https:
[//drive.google.com/file/d/1m33Bf5FJT0r0q21xJ7RcKgt9E8qJdwQp/view?usp=sharing](https://drive.google.com/file/d/1m33Bf5FJT0r0q21xJ7RcKgt9E8qJdwQp/view?usp=sharing)

4 Conclusion

The project works: the UR10e captures a maze it has never seen, finds it, solves it, and draws the solution with a pen, fully autonomously. The main lesson was that the planning and vision stages were not the hard part. The hard part was execution: the rate and the way commands and feedback move between the controller and the robot dominated the quality of the motion. Problems that looked like control tuning issues (jitter, a shrunken drawing) were actually caused by a 10 Hz feedback stream and by how the velocity streaming interface handles each new command. Recording telemetry of what the controller actually saw and commanded at every cycle is what made these problems findable.

Remaining shortcomings: the maze sheet must stay where it was during capture, since the tag-to-base transform is measured by hand; the pen height is a fixed constant rather than sensed; and the system assumes the maze has exactly two openings on the outer boundary.

5 Possible Improvements